

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
pd.options.display.max_columns=99
```

```
df=pd.read_csv("/content/Wholesale customers data.csv")
```

```
df.head()
```

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	
0	2	3	12669	9656	7561	214	2674	1338	
1	2	3	7057	9810	9568	1762	3293	1776	
2	2	3	6353	8808	7684	2405	3516	7844	
3	1	3	13265	1196	4221	6404	507	1788	
4	2	3	22615	5410	7198	3915	1777	5185	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
df.tail()
```

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	
435	1	3	29703	12051	16027	13135	182	2204	
436	1	3	39228	1431	764	4510	93	2346	
437	2	3	14531	15488	30243	437	14841	1867	
438	1	3	10290	1981	2232	1038	168	2125	
439	1	3	2787	1698	2510	65	477	52	

```
df.shape
```

```
(440, 8)
```

df.describe()

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
count	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000
mean	1.322727	2.543182	12000.297727	5796.265909	7951.277273	3071.931818	2881.493182	1524.870455
std	0.468052	0.774272	12647.328865	7380.377175	9503.162829	4854.673333	4767.854448	2820.105937
min	1.000000	1.000000	3.000000	55.000000	3.000000	25.000000	3.000000	3.000000
25%	1.000000	2.000000	3127.750000	1533.000000	2153.000000	742.250000	256.750000	408.250000
50%	1.000000	3.000000	8504.000000	3627.000000	4755.500000	1526.000000	816.500000	965.500000
75%	2.000000	3.000000	16933.750000	7190.250000	10655.750000	3554.250000	3922.000000	1820.250000
max	2.000000	3.000000	112151.000000	73498.000000	92780.000000	60869.000000	40827.000000	47943.000000



df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Channel         440 non-null   int64
1   Region          440 non-null   int64
2   Fresh           440 non-null   int64
3   Milk            440 non-null   int64
4   Grocery         440 non-null   int64
5   Frozen          440 non-null   int64
6   Detergents_Paper 440 non-null   int64
7   Delicassen      440 non-null   int64
dtypes: int64(8)
memory usage: 27.6 KB
```

plt.style.use("fivethirtyeight")

```
x_question=df.iloc[:,6:35]
q_mean=x_question.mean(axis=0)
total_mean=q_mean.mean
```

```
q_mean_reset_df = q_mean.reset_index(level=0)
q_mean_reset_df.head()
```

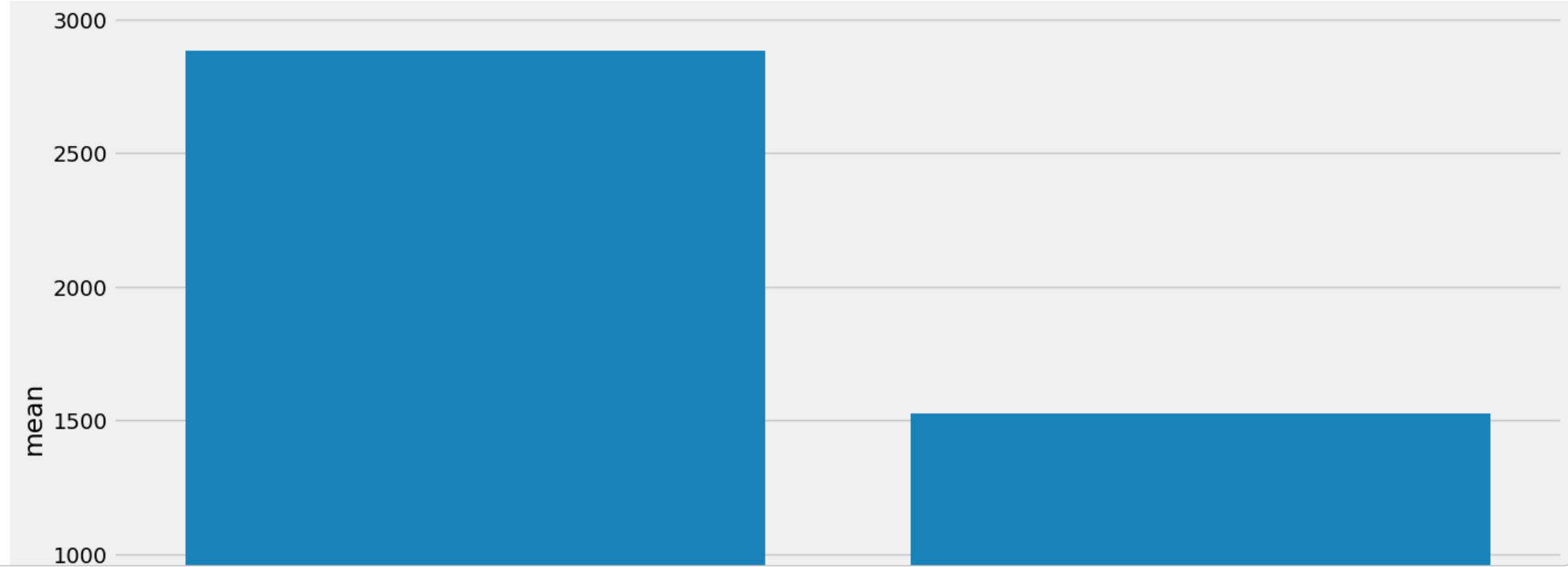
	index	0
0	Detergents_Paper	2881.493182
1	Delicassen	1524.870455

Next steps:

[Generate code with q_mean_reset_df](#)[New interactive sheet](#)

```
plt.figure(figsize=(15,9))
q_mean_df = q_mean.to_frame(name='mean').reset_index()
sns.barplot(x="index",y='mean',data=q_mean_df)
```

<Axes: xlabel='index', ylabel='mean'>



```
corr=df.corr()
plt.figure(figsize=(15,9))
sns.heatmap(corr,annot=True,cmap="coolwarm")
```

<Axes: >



```
x=df.iloc[:,6:35]
```

```
from sklearn.decomposition import PCA
pca=PCA(n_components=2,random_state=42)
x_pca=pca.fit_transform(x)
```

```
x_pca
np.cumsum(pca.explained_variance_ratio_)[1]
```

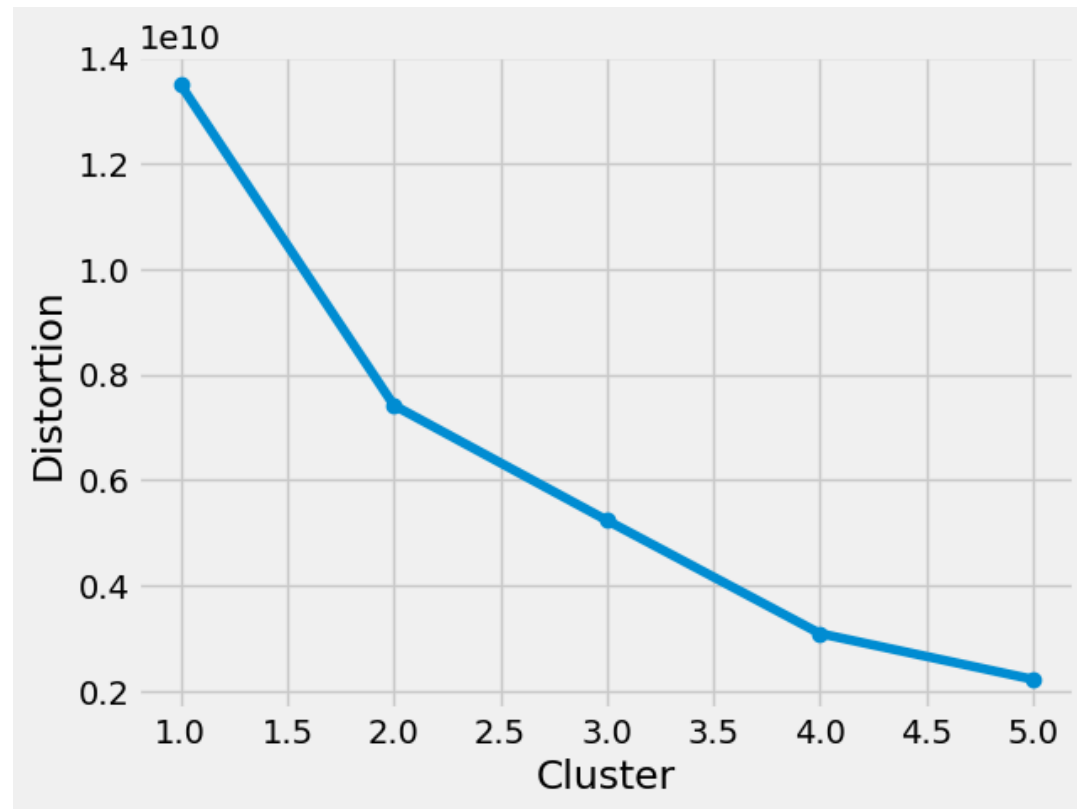
```
np.float64(0.9999999999999999)
```

```
from sklearn.cluster import KMeans
distortion=[]
cluster_range=range(1,6)
for i in cluster_range:
    model=KMeans(n_clusters=i,random_state=42,n_init=10)
```

```
model.fit(x_pca)
distortion.append(model.inertia_)
```

```
from sklearn.cluster import KMeans
distortion=[]
cluster_range=range(1,6)
for i in cluster_range:
    model=KMeans(n_clusters=i,random_state=42,n_init=10)
    model.fit(x_pca)
    distortion.append(model.inertia_)
```

```
plt.plot(cluster_range,distortion,marker="o")
plt.xlabel("Cluster")
plt.ylabel("Distortion")
plt.show()
```



```
from sklearn.cluster import KMeans
model=KMeans(n_clusters=3,init='k-means++',random_state=42,n_init=10)
model.fit(x_pca)
v=model.predict(x_pca)
```

```
x = df.iloc[:,6:35]

from sklearn.decomposition import PCA
pca = PCA(n_components=2,random_state=42)
x_pca = pca.fit_transform(x)

from sklearn.cluster import KMeans
model = KMeans(n_clusters=3,init='k-means++',random_state=42,n_init=10)
model.fit(x_pca)
y = model.predict(x_pca)

plt.scatter(x_pca[:,0],x_pca[:,1],c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

